

FOOD STORE

Sistema de Gestión de Pedidos de Comida

Especificación Técnica del Sistema

Versión 6.0 · Feature-Firstre

Materia	Programación 4
Carrera	Tecnicatura Universitaria en Programación (TUP)
Modalidad	Trabajo Práctico Integrador (TPI)
Stack	React + TypeScript + FastAPI + PostgreSQL + WebSocket + Cloudinary
Versión doc.	6.0 — UML v7 + WebSocket + Cloudinary + Feature-First

1. Visión General del Sistema

Food Store es una aplicación web full-stack para la gestión integral de un negocio de comidas. Permite a los clientes explorar el catálogo, agregar productos al carrito, realizar pedidos con pago integrado vía MercadoPago y hacer seguimiento en tiempo real del estado de su pedido mediante WebSocket. Los administradores gestionan el catálogo (con imágenes gestionadas por Cloudinary), el stock, los pedidos y los usuarios desde un panel centralizado.

1.1 Objetivos del Sistema

#	Actor	Objetivo principal
OBJ-01	Cliente	Navegar el catálogo, gestionar carrito, pagar con MercadoPago y rastrear pedidos en tiempo real vía WebSocket.
OBJ-02	Administrador	Gestionar categorías, productos (con imágenes Cloudinary), stock y ciclo de vida de pedidos.
OBJ-03	Gestor de Stock	Controlar disponibilidad y cantidad de stock.
OBJ-04	Gestor de Pedidos	Avanzar estados de pedidos según la máquina de estados FSM definida.
OBJ-05	Sistema	Visualizar cambios de estado en tiempo real a clientes y admins vía WebSocket.
OBJ-06	Sistema	Gestionar imágenes de productos y categorías en Cloudinary.
OBJ-07	Sistema	Garantizar trazabilidad completa de transiciones de estado mediante audit trail append-only.
OBJ-08	Sistema	Procesar y registrar pagos a través de la pasarela MercadoPago.

1.2 Alcance v6.0

- Autenticación y autorización con JWT y RBAC (4 roles) + invalidación de refresh token
- Catálogo de productos con categorías jerárquicas, ingredientes (es_alergeno) y unidades de medida
- Imágenes de productos y categorías gestionadas por Cloudinary (upload, transformaciones, eliminación)
- Carrito de compras con persistencia mediante Zustand + localStorage
- Gestión de pedidos con máquina de estados de 5 estados y audit trail append-only
- Pasarela de pagos MercadoPago Checkout PRO: tarjeta de crédito/débito, Dinero en cuenta
- WebSocket en tiempo real para cambios de estado de pedidos
- Webhook de MercadoPago para confirmación automática de pagos
- Módulo DireccionEntrega: CRUD completo con dirección principal por usuario
- Panel de administración: dashboard con graficos, CRUD de entidades, gestión de pedidos y stock
- Rate limiting: máximo 5 intentos fallidos por IP en 15 minutos en el login/register
- CORS configurado correctamente con CORSMiddleware para la separación frontend/backend
- Seed data obligatorio: roles, estados de pedido, formas de pago, unidades de medida y usuario admin
- API REST documentada con FastAPI/OpenAPI — accesible en /docs y /redoc

1.3 Stack Tecnológico

Capa	Tecnología	Versión	Rol en el sistema
Frontend	React + TypeScript	18.x + 5.x	UI, enrutamiento, componentes
Frontend	Vite	5.x	Build tool y dev server
Frontend	Tailwind CSS	3.x	Estilos utility-first
Frontend	TanStack Query	5.x	Fetching, caché y sincronización de datos del servidor
Frontend	TanStack Form	0.x	Gestión de formularios con validación
Frontend	Zustand	4.x	Estado global del cliente (carrito, sesión, pagos, WS, UI)
Frontend	Axios	1.x	Cliente HTTP con interceptors JWT
Frontend	recharts, react-chartjs-2, etc...	2.x	Gráficos del dashboard de administración
Frontend	@mercadopago/sdk-react	—	SDK oficial MercadoPago para tokenización PCI-compliant
Backend	FastAPI	0.111+	Framework REST + WebSocket + generación automática OpenAPI
Backend	SQLModel	0.0.19+	ORM + schemas Pydantic integrados
Backend	PostgreSQL	15+	Base de datos relacional
Backend	Passlib (bcrypt)	—	Hashing de contraseñas (cost factor ≥ 12)
Backend	mercadopago	2.3.0+	SDK oficial MercadoPago Python
Backend	cloudinary	1.x+	SDK Python para upload y gestión de imágenes

2. Arquitectura del Sistema

2.1 Capas del Backend — Flujo de Dependencias

El backend aplica una arquitectura de capas con módulos por feature. El patrón Unit of Work (UoW) se ubica entre la capa de servicio y los repositorios, garantizando atomicidad transaccional. El WebSocket Manager (WSManager) reside en el core y es invocado por la capa de servicio para emitir notificaciones después del commit. El flujo de dependencias es unidireccional y no puede invertirse.

Figura 1 — Capas del backend y flujo de dependencias unidireccional

Regla de oro — flujo de imports: Router → Service → UoW → Repository → Model WSManager es invocado por Service DESPUÉS del commit (fuera del bloque UoW). Ninguna capa puede importar de la capa superior.

Capa	Archivo de referencia	Responsabilidad	Conoce a
Router	router.py	HTTP puro: parsear request, validar schema Pydantic, delegar al Service, serializar response.	Service
Service	service.py	Lógica de negocio: stateless, orquesta repos a través del UoW. Emite eventos WS post-commit.	UoW, WSManager
Unit of Work	core/uow.py	Gestión de transacción: sesión de BD, acceso a repositorios, commit/rollback automático.	Repository, Session
WS Manager	core/websocket.py	Gestión del pool de conexiones WebSocket. Broadcast a suscriptores por pedido_id o canal admin.	WebSocket Connections
Repository	repository.py	Acceso a BD: queries sin lógica de negocio. Hereda de BaseRepository[T].	Model, Session
Model	model.py	SQLModel tables + relaciones. Sin imports de capas superiores.	Ninguna

2.2 Capas del Frontend — Feature-Sliced Design

El frontend aplica Feature-Sliced Design. Cada feature es autocontenida. Los imports fluyen de arriba hacia abajo: Pages → Features → Hooks/Stores → API → Types. El hook useOrderStatus encapsula la conexión WebSocket y se comunica con el orderStatusStore de Zustand.

Figura 2 — Capas del frontend con WebSocket y Cloudinary

Separación de responsabilidades frontend: Zustand gestiona estado del CLIENTE: carrito, sesión, proceso de pago, estado WS, UI local. TanStack Query gestiona estado del SERVIDOR: productos, pedidos, dashboard. Cloudinary Upload Widget gestiona el ciclo de vida de imágenes fuera del form.

2.3 Módulos Backend (Feature-First)

Módulo	Ruta	Descripción
auth	app/modules/auth/	Login, registro, refresh, logout. JWT access (30 min) + refresh (7 días). Rate limiting.
usuarios	app/modules/usuarios/	CRUD usuarios + asignación de roles RBAC. Soft delete.
direcciones	app/modules/direcciones/	CRUD completo DireccionEntrega por usuario. PATCH /principal.

categorias	app/modules/categorias/	Categorías jerárquicas con CTE recursiva + gestión de imagen Cloudinary. Soft delete.
productos	app/modules/productos/	Catálogo con Ingrediente, UnidadMedida e imagenes_url[] vía Cloudinary. Stock.
pedidos	app/modules/pedidos/	Dominio central: FSM, audit trail, historial append-only. Emite eventos WS post-commit.
pagos	app/modules/pagos/	Integración MercadoPago: crear pago, webhook IPN, registro de transacciones.
uploads	app/modules/uploads/	Upload y eliminación de imágenes en Cloudinary. Devuelve URL + public_id.
ws	app/core/ws_manager.py	WebSocket Manager: pool de conexiones, broadcast por pedido_id y canal admin.
admin	app/modules/admin/	Dashboard con métricas, gestión de stock y usuarios desde el panel.

3. Modelo de Datos — UML v7

El esquema aplica Tercera Forma Normal (3FN), Soft Delete (deleted_at TIMESTAMPTZ), Snapshot Pattern en pedidos y Audit Trail append-only en HistorialEstadoPedido. La versión 7 incorpora: Cloudinary en imágenes de Producto y Categoría (imagenes_url TEXT[], imagen_url TEXT), la nueva entidad UnidadMedida, 5 estados de pedido (se elimina EN_CAMINO) y el campo expires_at en UsuarioRol.

LINK: 📄 food_store_erd_v7.svg

3.1 Dominio 1 — Identidad y Acceso

Entidad	Campo clave	Tipo	Restricción	Notas
Usuario	id	BIGSERIAL	PK	Soft-delete vía deleted_at
Usuario	email	VARCHAR(254)	UQ, NN	Validar con EmailStr (Pydantic v2)
Usuario	celular	VARCHAR(20)	NULL	Campo nuevo en v7
Usuario	password_hash	CHAR(60)	NN	bcrypt cost≥12. NUNCA almacenar plaintext
Rol	codigo	VARCHAR(20)	PK (semántica)	ADMIN STOCK PEDIDOS CLIENT
UsuarioRol	(usuario_id, rol_codigo)	BIGINT + VARCHAR	PK compuesta	Pivot N:M. Incluye asignado_por_id
UsuarioRol	expires_at	TIMESTAMPTZ	NULL	Nuevo v7 — rol temporal opcional
DireccionEntrega	alias	VARCHAR(50)	NULL	Ej: 'Casa', 'Trabajo'
DireccionEntrega	es_principal	BOOLEAN	NN, default false	Solo una por usuario

3.2 Dominio 2 — Catálogo de Productos

Entidad	Campo clave	Tipo	Restricción	Notas
Categoría	parent_id	BIGINT	FK self-ref, NULL	Jerarquía recursiva. ON DELETE SET NULL. CTE.
Categoría	imagen_url	TEXT	NULL	URL de imagen en Cloudinary
Producto	precio_base	DECIMAL(10,2)	CHECK ≥ 0, NN	Snapshot al crear pedido
Producto	imagenes_url	TEXT[]	NULL	Array de URLs Cloudinary (múltiples imágenes)
Producto	unidad_venta_id	BIGINT	FK → UnidadMedida.id, NULL	Nueva v7 — ej: kg, unidad, litro
Producto	stock_cantidad	INTEGER	CHECK ≥ 0, NN, default 0	Gestionado por rol STOCK
Producto	disponible	BOOLEAN	NN, default true	Toggle manual independiente del stock
Ingrediente	nombre	VARCHAR(100)	UQ, NN	Especificación completa en v7

Ingrediente	stock_cantidad	INTEGER	NN, CHECK ≥ 0 , DEFAULT 0	Nuevo v7 — stock del ingrediente
Ingrediente	es_alergeno	BOOLEAN	NN, default false	Badge de alérgenos en UI
UnidadMedida	id	BIGSERIAL	PK	Entidad nueva en v7
UnidadMedida	nombre	VARCHAR(50)	UQ, NN	Ej: kilogramo, litro, unidad
UnidadMedida	simbolo	VARCHAR(10)	UQ, NN	Ej: kg, L, ud — mostrado en ProductCard
UnidadMedida	tipo	VARCHAR(20)	NN	Ej: peso, volumen, contable
ProductoCategoria	(producto_id, cat_id)	BIGINT×2	PK compuesta	Pivot N:M. es_principal.
ProductoIngrediente	es_removible	BOOLEAN	NN	Habilita personalización del pedido
ProductoIngrediente	cantidad	DECIMAL(10,3)	NN, CHECK > 0	Nuevo v7 — cantidad del ingrediente
ProductoIngrediente	unidad_medida_id	BIGINT	FK → UnidadMedida.id, NN	Nuevo v7
FormaPago	codigo	VARCHAR(20)	PK semántica	MERCADOPAGO EFECTIVO TRANSFERENCIA

3.3 Dominio 3 — Ventas, Pagos y Trazabilidad

Entidad	Campo clave	Tipo	Restricción	Notas
EstadoPedido	codigo	VARCHAR(20)	PK semántica	Catálogo. Ver máquina de estados.
EstadoPedido	es_terminal	BOOLEAN	NN	true = no admite transiciones salientes
Pedido	estado_codigo	VARCHAR(20)	FK → EstadoPedido	Estado actual del pedido
Pedido	subtotal	DECIMAL(10,2)	NN, snap	Nuevo v7 — suma de ítems sin descuento
Pedido	descuento	DECIMAL(10,2)	NN, default 0.00, snap	Nuevo v7 — descuento aplicado
Pedido	costo_envio	DECIMAL(10,2)	NN, default 50.00	Valor fijo v1.
Pedido	total	DECIMAL(10,2)	CHECK ≥ 0 , NN	subtotal - descuento + costo_envio
DetallePedido	nombre_snapshot	VARCHAR(200)	NN, snap	Snapshot: nombre al crear. Inmutable.
DetallePedido	precio_snapshot	DECIMAL(10,2)	NN, snap	Snapshot: precio al crear. Inmutable.
DetallePedido	subtotal_snap	DECIMAL(10,2)	NN, snap	Nuevo v7 — precio_snapshot × cantidad

DetallePedido	personalizacion	INTEGER[]	NULL	IDs de ingredientes removidos
HistorialEstadoPedido	estado_desde	VARCHAR(20)	FK, NULL	NULL = transición inicial (RN-02)
HistorialEstadoPedido	estado_hacia	VARCHAR(20)	FK → EstadoPedido.codigo, NN	Estado destino de la transición
HistorialEstadoPedido	created_at	TIMESTAMPTZ	NN, append-only	Nunca updated_at. Append-only (RN-03).
Pago	mp_payment_id	BIGINT	UQ, NULL	ID devuelto por MercadoPago
Pago	mp_status	VARCHAR(30)	NN	pending / approved / rejected
Pago	mp_status_detail	VARCHAR(100)	NULL	Nuevo v7 — detalle del estado MP
Pago	transaction_amount	DECIMAL(10,2)	NN	Nuevo v7 — monto cobrado por MP
Pago	payment_method_id	VARCHAR(50)	NULL	Nuevo v7 — método usado (visa, master...)
Pago	external_reference	VARCHAR(100)	UQ, NN	UUID del Pedido como referencia MP
Pago	idempotency_key	VARCHAR(100)	UQ, NN	UUID generado por backend. Evita cobros duplicados.

3.4 Máquina de Estados — Pedido (v7: 5 estados)

Figura 3 — Máquina de estados del Pedido (FSM) v7. Se elimina EN_CAMINO de la v5.

Código	Descripción	Orden	es_terminal	Transiciones válidas
PENDIENTE	Pedido creado, pago pendiente	1	false	→ CONFIRMADO, → CANCELADO
CONFIRMADO	Pago procesado y confirmado	2	false	→ EN_PREP, → CANCELADO
EN_PREP	En preparación en cocina	3	false	→ ENTREGADO, → CANCELADO (solo ADMIN/PEDIDOS)
ENTREGADO	Entrega confirmada	4	TRUE ✓	— (estado terminal)
CANCELADO	Pedido cancelado	5	TRUE ✓	— (estado terminal)

Reglas de negocio — Pedidos: RN-01: Un estado con es_terminal = true no admite transiciones salientes. Validación en Service. RN-02: El primer registro de HistorialEstadoPedido siempre tiene estado_desde = NULL. RN-03: La tabla HistorialEstadoPedido es append-only. Ninguna capa puede emitir UPDATE ni DELETE. RN-04: El total, nombre y precio en DetallePedido son un snapshot inmutable al crear el pedido. RN-05: El motivo es obligatorio si nuevo_estado = CANCELADO. RN-06: Al completar avanzar_estado() con éxito, el Service llama WSManger.broadcast() FUERA del bloque UoW.

4. Autenticación y Autorización

4.1 Flujo de Autenticación

Paso	Actor	Acción	Resultado esperado
1	Cliente	POST /api/v1/auth/login con email + password	HTTP 200 + access token (30 min) + refresh token (7 días)
2	Frontend	Almacena access token en cookies only http	Token disponible para interceptor Axios
4	Backend	Dependency get_current_user() valida JWT y carga el usuario	Objeto usuario inyectado en el handler
5	Backend	require_role([Rol.ADMIN]) verifica roles del token	HTTP 403 si rol insuficiente
6	Cliente	POST /api/v1/auth/refresh con refresh token	Nuevo access token sin requerir re-login
7	Cliente	POST /api/v1/auth/logout	Remover cookies en el front y desloguear al usuario

4.2 Roles y Permisos (RBAC)

Rol	Código	Permisos principales	Restricciones
Administrador	ADMIN	CRUD completo: usuarios, categorías, productos (imágenes Cloudinary), pedidos, stock.	Sin restricciones.
Gestor de Stock	STOCK	Leer productos, actualizar stock_cantidad y disponible, ver ingredientes.	Sin acceso a usuarios ni datos financieros.
Gestor de Pedidos	PEDIDOS	Ver todos los pedidos, avanzar estados CONFIRMADO → EN_PREPARACION → ENTREGADO, ver historial.	Sin acceso a productos ni finanzas.
Cliente	CLIENT	Ver catálogo, gestionar carrito, crear pedidos, ver sus propios pedidos.	Solo accede a sus propios datos.

4.3 Rate Limiting en Autenticación

Configuración	Valor
Límite	5 intentos fallidos por dirección IP en 15 minutos
Endpoint protegido	Login y Registro
Respuesta al superar	HTTP 429 Too Many Requests con header Retry-After

5. Especificación de API REST

Todos los endpoints usan el prefijo `/api/v1`. Los errores siguen RFC 7807 (Problem Details). La documentación interactiva se genera automáticamente en `/docs` (Swagger UI) y `/redoc`. Los endpoints WebSocket se documentan por separado en la Sección 9.

Convenciones globales: Error estándar RFC 7807: { "detail": "mensaje", "code": "ERROR_CODE", "field": "campo_opcional" } Paginación: GET `/recursos?page=1&size=20` → { "items": [...], "total": N, "page": 1, "size": 20, "pages": P } Soft delete: todos los GET filtran WHERE `deleted_at IS NULL`.

5.1 Módulo Auth

Método	Endpoint	Body / Params	Response	Auth requerida
POST	<code>/api/v1/auth/register</code>	{ nombre, apellido, email, password }	201 UserResponse	No
POST	<code>/api/v1/auth/login</code>	{ email, password }	200 TokenResponse	No — rate limited 5/15min
POST	<code>/api/v1/auth/refresh</code>	{ refresh_token }	200 TokenResponse	No
POST	<code>/api/v1/auth/logout</code>	{ refresh_token }	204 No Content	Bearer token
GET	<code>/api/v1/auth/me</code>	—	200 UserResponse	Bearer token

5.2 Módulo Productos

Método	Endpoint	Descripción	Rol requerido	Response
GET	<code>/api/v1/productos</code>	Listar (filtro: categoria, disponible, search, page, size)	Público	200 PaginatedProductos
GET	<code>/api/v1/productos/{id}</code>	Detalle con ingredientes, categorías, unidades y stock	Público	200 ProductoDetail
POST	<code>/api/v1/productos</code>	Crear producto con imagenes_url[], unidad_venta_id	ADMIN	201 ProductoRead
PUT	<code>/api/v1/productos/{id}</code>	Actualizar producto	ADMIN	200 ProductoRead
PATCH	<code>/api/v1/productos/{id}/disponibilidad</code>	Cambiar disponible (true/false)	ADMIN, STOCK	200 ProductoRead
PATCH	<code>/api/v1/productos/{id}/imagenes</code>	Actualizar lista imagenes_url[] del producto	ADMIN	200 ProductoRead
DELETE	<code>/api/v1/productos/{id}</code>	Soft delete producto	ADMIN	204 No Content
GET	<code>/api/v1/productos/{id}/ingredientes</code>	Listar ingredientes del producto	Público	200 List[IngredienteRead]
POST	<code>/api/v1/productos/{id}/ingredientes</code>	Asociar ingrediente con cantidad y unidad	ADMIN	201 ProductoIngredienteRead

5.3 Módulo Pedidos

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/pedidos	Listar propios (CLIENT) o todos (ADMIN/PEDIDOS)	CLIENT/ADMIN/PEDIDOS	200 PaginatedPedidos
GET	/api/v1/pedidos/{id}	Detalle completo con líneas, trazabilidad y pago	Propietario/ADMIN	200 PedidoDetail
POST	/api/v1/pedidos	Crear pedido desde carrito. Todo en una transacción (UoW).	CLIENT	201 PedidoRead
PATCH	/api/v1/pedidos/{id}/estado	Avanzar estado. Valida FSM. UoW atómico. Notifica WS post-commit.	ADMIN/PEDIDOS	200 PedidoRead
GET	/api/v1/pedidos/{id}/historial	Historial completo. ORDER BY created_at ASC.	Propietario/ADMIN	200 List[HistorialRead]
DELETE	/api/v1/pedidos/{id}	Cancelar propio (solo PENDIENTE o CONFIRMADO).	CLIENT propietario	200 PedidoRead

5.4 Módulo Pagos (MercadoPago)

Método	Endpoint	Descripción	Rol requerido	Response
POST	/api/v1/pagos/crear	Crea pago con token de tarjeta. Registra en tabla Pago.	CLIENT	201 PagoResponse
POST	/api/v1/pagos/webhook	Endpoint IPN de MercadoPago. Actualiza estado del pago y del pedido. Notifica WS.	Público (validar firma)	200 { status: ok }
GET	/api/v1/pagos/{pedido_id}	Consulta el pago asociado a un pedido.	Propietario/ADMIN	200 PagoResponse

5.5 Módulo Uploads — Cloudinary (Nuevo v6.0)

Método	Endpoint	Descripción	Rol requerido	Response
POST	/api/v1/uploads/imagen	Sube una imagen a Cloudinary. Recibe multipart/form-data. Devuelve secure_url y public_id.	ADMIN	201 CloudinaryResponse
DELETE	/api/v1/uploads/imagen/{public_id}	Elimina una imagen de Cloudinary por su public_id. Usar al borrar producto o categoría.	ADMIN	204 No Content

6. Schemas de Request / Response (Pydantic v2)

Todos los schemas usan Pydantic v2. Se definen schemas separados para Create, Update y Read. Nunca se expone el model de SQLAlchemy directamente como response.

6.1 Auth

Schema	Campos requeridos	Validaciones
LoginRequest	email: EmailStr, password: str	password mínimo 8 caracteres
RegisterRequest	nombre, apellido, email: EmailStr, password: str	nombre/apellido min 2 max 80. password min 8. Unicidad de email en servicio.
TokenResponse	access_token, refresh_token, token_type, expires_in: int	token_type = 'bearer'. expires_in en segundos.
UserResponse	id, nombre, apellido, email, roles: list[str], created_at	Nunca incluye password_hash.

6.2 Pedidos

Schema	Campos	Validaciones / Notas
CrearPedidoRequest	items: list[ItemPedidoRequest], forma_pago_codigo: str, direccion_id: int None, notas: str None	Mínimo 1 item. forma_pago_codigo debe existir en catálogo.
ItemPedidoRequest	producto_id: int, cantidad: int, personalizacion: list[int] None	cantidad ≥ 1. personalizacion = IDs de ingredientes removidos.
AvanzarEstadoRequest	nuevo_estado: str, motivo: str None	motivo obligatorio si nuevo_estado = CANCELADO (RN-05).
PedidoRead	id, estado_codigo, subtotal, descuento, costo_envio, total, created_at	Versión compacta para listados. Incluye subtotal y descuento (v7).
PedidoDetail	id, estado_codigo, subtotal, descuento, costo_envio, total, items, historial, pago	Versión completa para vista de detalle.
DetallePedidoRead	producto_id, nombre_snapshot, precio_snapshot, subtotal_snap, cantidad, personalizacion	Snapshot inmutable. subtotal_snap nuevo en v7.

6.3 Cloudinary

Schema	Campos	Notas
CloudinaryResponse	secure_url: str, public_id: str, width: int, height: int, format: str, resource_type: str	Respuesta del upload. secure_url se almacena en imagenes_url[] de Producto o imagen_url de Categoría.
ImagenProductoUpdate	imagenes_url: list[str]	Lista completa de URLs. Reemplaza el array anterior.
ImagenCategoriaUpdate	imagen_url: str None	URL única de Cloudinary para la categoría.

7. Patrón Unit of Work (UoW)

El Unit of Work actúa como director de orquesta que garantiza que todas las operaciones de base de datos dentro de una transacción de negocio tengan éxito o fallen como un conjunto. El commit ocurre en el UoW, no en el service. Las notificaciones WebSocket se emiten DESPUÉS del commit exitoso, fuera del bloque UoW.

7.1 Flujo de una Operación con UoW — Crear Pedido

Figura 4 — Flujo de creación de pedido con Unit of Work. Todos los INSERT son atómicos.

Paso	Capa	Operación	¿Toca BD?
1	Router	Recibe POST /api/v1/pedidos. Valida body con CrearPedidoRequest.	No
2	Router	Abre contexto: with UnitOfWork() as uow: — llama service.crear_pedido(uow, body, usuario_id).	No
3	Service	Itera items. Para cada uno: uow.productos.get_by_id(). Verifica disponible = true.	Lectura
4	Service	Calcula subtotal, descuento (si aplica) y total = subtotal - descuento + costo_envio.	No
5	Service	Llama uow.pedidos.create(pedido). uow.flush() → obtiene pedido.id.	INSERT + flush
6	Service	Crea DetallePedido por cada item con nombre_snapshot, precio_snapshot, subtotal_snap.	INSERT × N
7	Service	Crea primer HistorialEstadoPedido con estado_desde=None (RN-02).	INSERT
8	UoW	__exit__ sin excepción → session.commit(). Todo persiste atómicamente.	COMMIT
9	Router	Serializa pedido con PedidoRead.model_validate(pedido). Retorna HTTP 201.	No
ERR	UoW	Si cualquier paso 3-7 lanza excepción → __exit__ llama rollback(). Nada persiste.	ROLLBACK

7.2 Flujo Avanzar Estado con WebSocket

Figura 5 — Flujo de avanzar_estado con UoW + notificación WebSocket post-commit.

Paso	Capa	Operación	¿Toca BD/WS?
1	Router	Recibe PATCH /pedidos/{id}/estado. Valida AvanzarEstadoRequest.	No
2	Service	Dentro de with UoW() as uow: — obtiene pedido, valida FSM.	Lectura
3	Service	UPDATE Pedido.estado_codigo = nuevo_estado.	UPDATE
4	Service	INSERT HistorialEstadoPedido con estado_desde, estado_hacia, usuario_id, motivo.	INSERT
5	UoW	__exit__ sin excepción → session.commit(). Cambio de estado persiste.	COMMIT
6	Service	FUERA del bloque UoW: await ws_manager.broadcast_pedido(pedido_id, evento)	WebSocket

7	WSManager	Itera conexiones activas para pedido_id y canal /ws/admin/pedidos. Envía JSON.	WS Send
8	Router	Serializa PedidoRead y retorna HTTP 200.	No

7.3 BaseRepository[T] Genérico

Método	Descripción
get_by_id(entity_id: int) → T None	Obtiene entidad por clave primaria. Retorna None si no existe.
list_all(skip: int, limit: int) → list[T]	Listado simple sin filtros.
count() → int	Cantidad total de registros. Útil para paginación.
create(entity: T) → T	Agrega a sesión + flush() + refresh(). Retorna entidad con ID asignado.
update(entity: T) → T	Agrega entidad modificada a sesión + flush() + refresh().
soft_delete(entity: T) → None	Asigna deleted_at = now(). Solo para entidades con soft-delete.
hard_delete(entity: T) → None	Hard delete. Solo se usa cuando el modelo no tiene soft-delete.

8. Integración MercadoPago

Food Store integra MercadoPago Checkout PRO. Permite procesar pagos con tarjeta de crédito/débito, Rapipago, Pago Fácil y Cuenta MercadoPago sin redirigir al cliente fuera del sitio.

¿Por qué Checkout PRO con Orders? - Única integración para múltiples medios de pago. - Datos de tarjeta tokenizados por MercadoPago.js — NUNCA pasan por el servidor de Food Store (PCI SAQ-A). - Notificaciones push (webhook) para confirmación asíncrona del pago. - idempotency_key UUID generado por el backend evita cobros duplicados por reintento.

9. WebSocket — Notificaciones en Tiempo Real

Food Store utiliza WebSocket nativo de FastAPI para mostrar a los clientes y administradores cuando el estado de un pedido cambia. Esto reemplaza el polling periódico y mejora la experiencia de usuario al mostrar actualizaciones instantáneas en la pantalla de seguimiento del pedido y en el panel de administración.

9.1 Arquitectura WebSocket

Figura 7 — Arquitectura WebSocket: WSManager en el core, canales de suscripción y flujo de broadcast.

Principios de diseño:

- El WSManager es un singleton en app/core/ws_manager.py. Se inyecta en los servicios via FastAPI Depends7.
- Autenticación via query param token=<jwt> en la URL del WebSocket.
- El broadcast SIEMPRE ocurre DESPUÉS del commit() del UoW, nunca dentro del bloque transaccional.
- Si no hay suscriptores, el broadcast es un no-op (silencioso).

9.2 Endpoints WebSocket

Endpoint	Tipo	Descripción	Auth	Payload recibido
WS /ws/pedidos	WebSocket	Feed de todos los pedidos. Recibe todos los cambios de estado.	JWT en ?token=(ADMIN/PEDIDOS)	JSON: { event, pedido_id, usuario_id,

				estado_nuevo, estado_anterior, timestamp }
--	--	--	--	--

9.3 WManager — Implementación Backend

Método	Descripción
connect(ws, pedido_id 'admin')	Registra una conexión WebSocket en el pool bajo el canal dado.
disconnect(ws, pedido_id 'admin')	Elimina la conexión del pool. Se llama en el bloque finally del endpoint WS.
broadcast_pedido(pedido_id, evento)	Envía JSON al cliente dueño del pedido y al canal admin. Captura errores de conexiones caídas.
broadcast_to_role(rol, evento)	Envía JSON a la room del rol.

9.4 Estructura del Evento WebSocket

Campo	Tipo	Descripción
event	str	Tipo de evento. Valores: 'estado_cambiado' 'pedido_cancelado' 'pago_confirmado'
pedido_id	int	ID del pedido afectado
estado_anterior	str null	Código del estado previo. null en creación inicial.
estado_nuevo	str	Código del nuevo estado (ej: 'CONFIRMADO', 'EN_PREP', 'ENTREGADO')
usuario_id	int null	ID del usuario que realizó la acción. null si fue el sistema (webhook MP).
motivo	str null	Motivo de cancelación si aplica (RN-05).
timestamp	str	ISO 8601 UTC. Ej: '2025-08-12T14:30:00Z'

9.5 Implementación Frontend

El hook useOrderStatusWS encapsula toda la lógica de conexión WebSocket y expone un estado reactivo.

Elemento	Archivo	Descripción
useOrderStatusWS	hooks/useOrderStatus.ts	Hook personalizado. Conecta al WS, gestiona reconexión exponencial, actualiza Zustand y React Query.

9.6 Reconexión y Resiliencia

Escenario	Estrategia
Token expirado	El hook detecta error 4001 (close code) y llama al interceptor de refresh antes de reconectar.
Servidor caído	Intentos máximos configurables (default 10). Muestra badge 'Sin conexión en tiempo real' en UI.
Datos desincronizados	Al reconectar, el frontend llama GET /api/v1/pedidos/{id} para obtener el estado actual del servidor.

10. Cloudinary — Gestión de Imágenes

Food Store utiliza Cloudinary para almacenar, transformar y servir imágenes de productos y categorías. Las imágenes se suben al backend vía el módulo /uploads, que las envía a Cloudinary y devuelve la URL segura para almacenar en la base de datos.

¿Por qué Cloudinary?

- CDN global con transformaciones on-the-fly (resize, crop, formato WebP automático).
- Eliminación de imágenes por public_id sin depender de la URL completa.
- URL firmada disponible para contenido privado si se requiere en el futuro.
- Modo de subida: BACKEND (signed upload con API key/secret). No se expone el secret al frontend.

10.1 Flujo de Subida de Imagen

Figura 8 — Flujo completo de upload de imagen vía el módulo /uploads al backend Cloudinary.

Paso	Actor	Acción	Resultado
1	Admin	Selecciona imagen en el formulario de producto o categoría.	Archivo disponible en el componente React.
2	Frontend	POST /api/v1/uploads/imagen con multipart/form-data (field: file, folder: 'productos').	Request autenticado con JWT.
3	Backend (Router)	Valida tipo MIME (image/jpeg, image/png, image/webp) y tamaño (max 5 MB).	HTTP 400 si inválido.
4	Backend (Service)	Llama cloudinary.uploader.upload(file_bytes, folder='foodstore/productos').	Imagen almacenada en Cloudinary.
5	Cloudinary	Retorna objeto con secure_url, public_id, width, height, format.	Respuesta JSON con metadatos.
6	Backend	Serializa CloudinaryResponse y retorna HTTP 201.	secure_url y public_id disponibles.
7	Frontend	Agrega secure_url al array imagenes_url del formulario de producto.	URL lista para guardar con el producto.
8	Admin	Guarda el producto. PUT /api/v1/productos/{id} con imagenes_url actualizado.	Producto guardado con imagen en BD.

10.2 Flujo de Eliminación de Imagen

Paso	Actor	Acción	Resultado
1	Admin	Hace clic en eliminar imagen de un producto.	Frontend conoce el public_id de la imagen.
2	Frontend	DELETE /api/v1/uploads/imagen/{public_id} (URL encode del public_id).	Request autenticado con JWT.
3	Backend	Llama cloudinary.uploader.destroy(public_id).	Imagen eliminada de Cloudinary CDN.
4	Backend	Retorna HTTP 204 No Content.	Confirmación de eliminación.
5	Frontend	Actualiza imagenes_url[] del producto (quita la URL eliminada). PATCH /api/v1/productos/{id}/imagenes.	Imagen eliminada de BD y CDN.

10.3 Configuración Backend (cloudinary SDK)

Configuración	Descripción
cloudinary.config(cloud_name, api_key, api_secret)	Inicializado en app/core/config.py al arrancar la app.
allowed_formats	['jpg', 'jpeg', 'png', 'webp'] — rechaza formatos no soportados desde el SDK.
overwrite=False	Evita sobrescribir imágenes existentes con el mismo public_id.
unique_filename=True	Cloudinary genera un public_id único si no se especifica uno.
resource_type='image'	Solo se aceptan imágenes (no videos ni raw files).

11. Módulo Estadísticas

El módulo de estadísticas provee KPIs y métricas del negocio exclusivamente al rol ADMIN. Todas las consultas son de solo lectura y se ejecutan contra las tablas existentes del modelo (Pedido, DetallePedido, Pago, Producto). No requiere nuevas tablas ni migraciones adicionales. Los datos son consumidos por los gráficos recharts del panel de administración.

11.1 Estructura del Módulo

Archivo	Responsabilidad
app/modules/estadisticas/router.py	Define los endpoints GET /api/v1/estadisticas/*. Sin lógica, delega al service.
app/modules/estadisticas/service.py	Lógica de cálculo de KPIs. Llama a los repositorys.
app/modules/estadisticas/schemas.py	Schemas Pydantic: ResumenResponse, VentasPeriodoItem, ProductoTopItem, PedidosEstadoItem, IngresosResponse.

11.2 Queries Clave — Repository

Método	Notas
get_ventas_periodo(desde, hasta, agrupacion)	Usa DATE_TRUNC de PostgreSQL. agrupacion puede ser 'day', 'week', 'month'.
get_productos_top(limit)	Usa subtotal_snap (snapshot immutable) para ingresos precisos.
get_pedidos_por_estado()	Simple GROUP BY sobre el estado actual de cada pedido.
get_resumen_kpis()	Cada KPI es una query separada. El service ensambla el ResumenResponse.
get_ingresos_por_forma_pago(desde, hasta)	Solo pedidos con pago aprobado. Forma de pago desde Pedido.forma_pago_codigo.

11.3 Visualización Frontend — recharts

Gráfico	Endpoint consumido	Componente recharts	Datos mapeados
Ventas por período	GET /estadisticas/ventas	LineChart + Line	eje X: periodo, eje Y: total_ventas y cantidad_pedidos (dos líneas)
Top productos	GET /estadisticas/productos-top	BarChart + Bar	eje X: nombre (truncado), eje Y: ingresos. Tooltip: cantidad_vendida.
Distribución por estado	GET /estadisticas/pedidos-por-estado	PieChart + Pie	name: estado_codigo, value: cantidad. Cell con color por estado.
Ingresos por forma de pago	GET /estadisticas/ingresos	BarChart horizontal	eje Y: forma_pago_codigo, eje X: total. Muestra cantidad en tooltip.
KPIs cards	GET /estadisticas/resumen	StatCard custom	4 cards: ventas hoy, ticket promedio, pedidos activos, mes actual.

Reglas de negocio — Estadísticas: EST-01: Nunca incluir pedidos con estado_codigo = CANCELADO en cálculos de ingresos o cantidades vendidas. EST-02: Usar subtotal_snap de DetallePedido para ingresos por producto (garantiza precios históricos correctos). EST-03: Solo contar pagos con mp_status = 'approved' al calcular ingresos confirmados. EST-04: Todos los montos devueltos deben ser DECIMAL(10,2). Nunca float nativo Python para dinero. EST-05: Las queries de período aceptan desde y hasta como date (no datetime). El filtro usa BETWEEN.

12. Gestión de Estado con Zustand

Zustand es la librería de gestión de estado global del frontend. Food Store v6.0 requiere cinco stores con responsabilidades claramente separadas, incluyendo el nuevo wsStore para gestionar el estado de las conexiones WebSocket.

Figura 9 — Los cinco stores Zustand y sus responsabilidades. Persistencia selectiva por store.

Store	Archivo	Estado que gestiona	Middlewar e	Persiste
authStore	store/authStore.ts	accessToken, usuario, isAuthenticated	persist	Sí — solo el accessToken
cartStore	store/cartStore.ts	items del carrito, cantidades, personalizaciones	persist	Sí — items completos

Buenas prácticas de consumo de stores: - Suscripción por slice: `const itemCount = useCartStore(s => s.itemCount())` — evita re-renders innecesarios. - Actions extraídas sin re-render: `const { addItem } = useCartStore()` - Nunca suscribirse al store completo sin selector: ❌ `const store = useCartStore()` - Acceso fuera de React (interceptores): `useAuthStore.getState().accessToken` - wsStore: las actions `connect()` y `disconnect()` son llamadas solo por los hooks `useOrderStatusWS` y `useAdminOrdersFeed`. - wsStore nunca es accedido por componentes directamente — solo a través de los hooks WS.

13. Tests con TestClient

Food Store utiliza pytest con el TestClient de FastAPI (basado en httpx) para tests de integración de los endpoints REST y WebSocket. Los tests cubren los flujos críticos: autenticación, ciclo de vida de pedidos, pagos, estadísticas y WebSocket.

13.1 Configuración — conftest.py

Fixture	Scope	Descripción
engine	session	Crea motor SQLite en memoria o PostgreSQL de test. Aplica create_all(). Se descarta al final de la sesión.
db_session	function	Session de SQLAlchemy limpia para cada test. Hace rollback automático al finalizar cada test.
client	function	TestClient de FastAPI. Sobreescribe la dependency get_db con db_session.
admin_headers	function	Loguea un usuario ADMIN. Retorna Cookie.
client_headers	function	Loguea un usuario CLIENT. Retorna Cookie.
pedidos_headers	function	Loguea un usuario PEDIDOS. Retorna Cookie..
producto_factory	function	Crea un Producto con stock disponible en la BD de test.
pedido_factory	function	Crea un Pedido en estado PENDIENTE con un DetallePedido. Acepta usuario_id y producto_id.

Estrategia de base de datos en tests: SQLite in-memory para velocidad.

13.2 Estructura de Archivos de Test

Archivo	Módulo testeado	Tests principales
tests/conftest.py	—	Fixtures globales: engine, db_session, client, factories, headers.
tests/test_auth.py	auth	register OK, login OK, login credenciales inválidas (401), logout + revocación, rate limit (429).
tests/test_pedidos.py	pedidos	crear pedido OK, stock insuficiente (400), avanzar estado válido, avanzar estado inválido (422), cancelar propio, historial append-only.
tests/test_estadisticas.py	estadisticas	resumen OK, ventas por período, productos top, pedidos por estado, ingresos (solo approved). Verifica que CANCELADO no suma.

13.3 Patrones de Test por Módulo

Módulo	Patrón de test	Qué verificar
Auth	Arrange: crear usuario. Act: POST /auth/login. Assert: status 200, access_token en body, refresh_token en body.	Token válido, token_type='bearer', expiración en expires_in.
Pedidos FSM	Arrange: crear pedido en PENDIENTE. Act: PATCH /pedidos/{id}/estado con CONFIRMADO. Assert: 200, estado_codigo='CONFIRMADO'.	Transición válida actualiza estado. Historial append-only tiene nuevo registro.
Pedidos FSM (inválido)	Arrange: pedido en ENTREGADO (terminal). Act: PATCH /pedidos/{id}/estado con EN_PREP. Assert: 422.	RN-01: estado terminal rechaza transiciones.
Estadísticas	Arrange: crear N pedidos con distintos estados y productos. Act: GET /estadisticas/resumen. Assert: ventas_hoy > 0, CANCELADO excluido.	EST-01/EST-02/EST-03 validadas en tests de integración.

14. Configuración y Setup

14.1 Variables de Entorno

Variable	Descripción	Valor ejemplo
DATABASE_URL	Conexión a PostgreSQL	postgresql://user:pass@localhost:5432/foodstore_db
SECRET_KEY	Clave secreta para firmar JWT (mín. 32 chars)	your-super-secret-key-min-32-chars
ALGORITHM	Algoritmo JWT	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Expiración del access token en minutos	30
REFRESH_TOKEN_EXPIRE_DAYS	Expiración del refresh token en días	7
CORS_ORIGINS	Orígenes permitidos (JSON array)	["http://localhost:5173"]
MP_ACCESS_TOKEN	Access Token de MercadoPago (backend)	TEST-xxxx
MP_PUBLIC_KEY	Public Key de MercadoPago (para el frontend)	TEST-xxxx
MP_NOTIFICATION_URL	URL del webhook IPN de MercadoPago	https://dominio-ngrok.com/api/v1/pagos/webhook
CLOUDINARY_CLOUD_NAME	Cloud name de la cuenta Cloudinary	mi-cloud-name
CLOUDINARY_API_KEY	API Key de Cloudinary (backend only, no exponer)	123456789012345
CLOUDINARY_API_SECRET	API Secret de Cloudinary (backend only, secreto)	abcdefghijklmn
VITE_API_URL	URL base del backend (Vite — frontend)	http://localhost:8000

14.2 Seed Data Obligatorio — `app/db/seed.py`

Entidad	Registros a insertar
Rol	ADMIN, STOCK, PEDIDOS, CLIENT — los cuatro roles del sistema RBAC
EstadoPedido	PENDIENTE, CONFIRMADO, EN_PREP, ENTREGADO, CANCELADO — con es_terminal correspondiente (5 estados)
FormaPago	MERCADOPAGO (habilitado), EFECTIVO (habilitado), TRANSFERENCIA (habilitado)
UnidadMedida	kg (peso), g (peso), L (volumen), ml (volumen), ud (contable), porciones (contable)
Usuario admin	admin@foodstore.com / Admin1234! — con rol ADMIN asignado. Contraseña debe cambiarse en producción.

15. Patrones Aplicados en el Proyecto

Patrón	Capa	Descripción
Repository Pattern	Backend	Abstracción del acceso a BD. BaseRepository[T] genérico. Facilita testing con mocks.
Unit of Work	Backend	Gestión de transacciones atómicas. El Service opera dentro del contexto UoW sin gestionar la sesión directamente.
Service Layer	Backend	Lógica de negocio centralizada, stateless. Consume el UoW. Independiente del framework.
Snapshot Pattern	Backend/BD	Precios y nombres de producto inmutables al crear el pedido. Garantiza integridad histórica.
Soft Delete	Backend/BD	deleted_at TIMESTAMPTZ — registros lógicamente eliminados. Nunca DELETE físico en entidades de negocio.
Audit Trail Append-Only	Backend/BD	HistorialEstadoPedido: solo INSERT, nunca UPDATE/DELETE (RN-03). Trazabilidad completa.
State Machine (FSM)	Backend	Transiciones del pedido validadas en la capa de servicio contra el mapa de transiciones permitidas.
Idempotent Payments	Backend	UUID como idempotency_key enviado a MercadoPago. Evita cobros duplicados por reintentos.
Connection Pool	Backend	WSManager mantiene un pool de WebSocket activos por canal. Limpia conexiones cerradas automáticamente.
CDN Upload	Backend/Frontend	Imágenes subidas a Cloudinary (CDN global). Backend gestiona el upload signed. Frontend usa las URLs servidas por CDN.
Feature-Sliced Design	Frontend	Organización por features con límites de importación claros. Cada feature es autocontenida.
Custom Hooks	Frontend	Encapsulan lógica de TanStack Query y WebSocket en hooks reutilizables por dominio.
Optimistic Updates	Frontend	Actualización inmediata de UI antes de confirmar respuesta del servidor. Rollback en error.
Webhook	Backend	MercadoPago notifica de forma asíncrona el resultado del pago. Evita polling constante.

16. Rúbrica de Corrección — v6.0

Puntaje total: 240 puntos. Corrección escrita + video de demostración obligatorio.

Puntaje total: 280 puntos. Corrección escrita + video de demostración obligatorio.

Criterio	Pts	Excelente	Bueno	Regular	Insuficiente
Backend — Estructura y Configuración	10	Capas router/service/uow/repository/model. Módulos por dominio. core/ separado. Alembic + seed. CORS + rate limiting.	Separación parcial. Seed incompleto.	Estructura plana o sin capas.	Sin estructura reconocible.
Backend — Modelo de Datos (ERD v7)	15	SQLModel correcto, constraints, soft-delete, snapshot, entidades completas (UnidadMedida, Ingrediente, Pago completo). subtotal+descuento en Pedido.	Correctos, faltan algunas entidades v7.	Básicos sin patrones avanzados.	Incorrectos o incompletos.
Backend — Unit of Work y Repository	15	UoW completo con context manager, commit/rollback automático. BaseRepository[T] genérico. Ningún service hace session.commit(). WS broadcast fuera del UoW.	UoW presente pero incompleto o WS dentro del bloque.	Sin UoW, transacciones manuales.	Sin gestión de transacciones.
Backend — Capa de Servicio	15	FSM 5 estados implementada. RN-01/02/03/05/06 validadas. Services stateless. avanzar_estado() emite broadcast WS post-commit.	Lógica presente pero RN-06 faltante.	Lógica básica sin validaciones.	Sin capa de servicio.
Backend — Controladores REST	15	Verbos HTTP correctos, rutas semánticas, status codes precisos. Prefijo /api/v1. Schemas Pydantic separados.	Verbos y rutas correctas, algunos status codes incorrectos.	Verbos incorrectos o rutas no semánticas.	Sin convenciones REST.
Backend — MercadoPago	15	SDK Python con idempotency_key UUID. Webhook que procesa topic=payment, avanza pedido y notifica WS. Tabla Pago completa (mp_status_detail, transaction_amount).	SDK configurado, sin idempotency_key o sin WS.	Integración parcial.	Sin integración MP.
Backend — WebSocket	20	WSManager con pool, broadcast por canal, autenticación JWT en handshake, broadcast post-commit. Endpoints /ws/pedidos/{id} y /ws/admin/pedidos.	WSManager presente, sin canal admin o sin auth.	WS básico sin pool ni reconexión.	Sin WebSocket.
Backend — Cloudinary	15	Módulo /uploads completo: upload (validar MIME + tamaño), destroy por public_id. SDK configurado. imagenes_url[] en Producto, imagen_url en Categoria.	Upload funcional, sin destroy o sin validación.	Upload básico sin integración al modelo.	Sin Cloudinary.
Backend — Estadísticas	15	5 endpoints implementados. Queries correctas con GROUP BY, SUM, DATE_TRUNC. EST-01/02/03/04/05 respetadas. Schemas Pydantic tipados.	3-4 endpoints funcionales, queries sin validar CANCELADO.	Solo endpoint resumen básico.	Sin módulo estadísticas.
Tests con TestClient	20	conftest.py con fixtures reutilizables. test_auth, test_pedidos, test_pagos, test_estadisticas, test_uploads, test_websocket. Mocks correctos. Cobertura ≥ 60%.	Tests presentes pero sin mocks o sin test_websocket. Cobertura 40-60%.	Solo tests de auth básicos. Cobertura < 40%.	Sin tests.

Frontend — Estructura y TypeScript	10	Feature-sliced: pages/features/components/hooks/store/api/types. Sin cross-imports. strict: true, no any.	Estructura presente, algunos módulos mezclados.	Estructura plana.	Sin estructura.
Frontend — Zustand	10	5 stores implementados y tipados. persist correcto. wsStore con connection status. Suscripción por slice.	4 stores correctos, wsStore faltante.	Solo authStore y cartStore básicos.	Sin Zustand.
Frontend — TanStack Query	15	useQuery/useMutation para todo el fetch. queryKeys descriptivos. Invalidación tras mutaciones. Interceptor refresh 401 automático. Invalidación en eventos WS.	TanStack Query presente, invalidación parcial o sin WS.	Fetch directo con useEffect.	Sin TanStack Query.
Frontend — WebSocket	20	useOrderStatusWS y useAdminOrdersFeed implementados. Reconexión exponencial. wsStore actualizado. Timeline en tiempo real. Badge sin polling.	Hooks WS presentes, sin reconexión o sin admin feed.	WS básico sin hook reutilizable.	Sin WebSocket frontend.
Frontend — Cloudinary	10	Upload de imágenes desde formulario admin. Transformaciones en ProductCard (f_auto, q_auto, c_fill). Eliminación desde UI con DELETE /uploads.	Upload funcional, sin transformaciones o sin delete.	Solo muestra URL hardcodeada.	Sin Cloudinary frontend.
Frontend — Estadísticas	10	4 gráficos recharts funcionales: LineChart ventas, BarChart productos, PieChart estados, BarChart ingresos. KPI cards. TanStack Query con refetch automático.	2-3 gráficos funcionales, sin todos los recharts.	Solo tabla de datos sin gráficos.	Sin visualización estadísticas.
Frontend — Funcionalidades Cliente	15	Catálogo con debounce/filtros/paginación/skeleton. Carrito persist. Checkout con CardPayment de MP. Timeline WS en tiempo real.	Funcional con algunas carencias.	Lista sin filtros ni paginación.	Sin funcionalidades.
Frontend — Panel Admin	15	Dashboard KPIs + recharts. CRUD categorías/productos con upload Cloudinary. Gestión pedidos con FSM y feed WS. Gestión stock.	CRUD funcional, falta alguna feature.	Solo visualización.	Sin panel admin.
UI/UX y Diseño	10	Sistema de diseño consistente. Mobile-first. Skeleton loaders, toasts, modales, badge de conexión WS en tiempo real, estados vacíos.	Diseño consistente con pequeñas inconsistencias.	Diseño básico sin sistema.	Sin diseño coherente.
Calidad de Código	10	snake_case/camelCase/PascalCase. Funciones < 50 líneas. SRP. Docstrings. JSDoc. README.md completo.	Nomenclatura correcta, algunas funciones largas.	Mezcla de convenciones.	Sin convenciones.

Escala de calificación:
 252-280 pts (90-100%) — EXCELENTE: Proyecto completo, profesional, con todas las capas y buenas prácticas.
 196-251 pts (70-89%) — BUENO: Proyecto funcional con pequeños ajustes o funcionalidades faltantes. 140-195 pts (50-69%) — REGULAR: Proyecto básico con errores o funcionalidades incompletas. 0-139 pts (0-49%) — INSUFICIENTE: Proyecto incompleto, no funcional o no sigue la especificación.
 ⚠ Penalización -30%: El proyecto que no corra localmente siguiendo el README.

17. Entrega del Proyecto

17.1 Checklist de Entrega

Ítem	Descripción	Estado
CE-01	Link a repositorios GitHub público en la entrega	☐ Pendiente
CE-02	README.md con instrucciones de setup funcionando en máquina limpia	☐ Pendiente
CE-03	.env.example completo con variables de MercadoPago, Cloudinary y WebSocket documentadas	☐ Pendiente
CE-05	python -m app.db.seed ejecuta correctamente y carga datos iniciales (incluye UnidadMedida)	☐ Pendiente
CE-06	pnpm i + pnpm dev	☐ Pendiente
CE-07	pip install -r requirements.txt + uvicorn app.main:app sin errores	☐ Pendiente
CE-08	Swagger UI (/docs) accesible con todos los endpoints documentados (incluye /uploads)	☐ Pendiente
CE-09	Pago de prueba con tarjeta sandbox MP funciona end-to-end y notifica vía WS	☐ Pendiente
CE-10	Unit of Work correctamente implementado (ningún service.session.commit() directo)	☐ Pendiente
CE-11	5 Zustand stores implementados, tipados y con persist correcto (incluye wsStore)	☐ Pendiente
CE-12	WebSocket: cambio de estado desde panel admin actualiza UI del cliente sin recargar	☐ Pendiente
CE-13	Cloudinary: subir imagen de producto desde panel admin y verla en el catálogo	☐ Pendiente
CE-15	Link a video demostración (10-15 min) en README	☐ Pendiente
CE-16	Repositorio público verificado con sesión cerrada	☐ Pendiente

Apéndice — Referencias y Recursos

Tecnología	URL
FastAPI	https://fastapi.tiangolo.com
FastAPI WebSockets	https://fastapi.tiangolo.com/advanced/websockets/
SQLModel	https://sqlmodel.tiangolo.com
Pydantic v2	https://docs.pydantic.dev
TanStack Query v5	https://tanstack.com/query
TanStack Form	https://tanstack.com/form
Zustand	https://zustand-demo.pmnd.rs
Tailwind CSS	https://tailwindcss.com/docs
recharts	https://recharts.org
MercadoPago Developers (AR)	https://www.mercadopago.com.ar/developers/es
MercadoPago SDK Python	https://github.com/mercadopago/sdk-python
MercadoPago SDK React	https://github.com/mercadopago/sdk-react
Cloudinary Docs	https://cloudinary.com/documentation
Cloudinary Python SDK	https://cloudinary.com/documentation/python_integration
Cloudinary React SDK	https://cloudinary.com/documentation/react_integration
Cloudinary Transformations	https://cloudinary.com/documentation/transformation_reference